



Dahu Verity Builder and Dahu Verity Indexer

Prepared by Chris Bartlett

10th January 2026

Version 1.0

For The National Archives

Prepared for:

Dahu contact:

Chris Owens

Company The National Archives
Company Kew
Address Richmond
 Surrey, TW9 4AD

Phone

Email Chris.Owens@nationalarchives.gov.uk

Chris Bartlett, Director

Dahu Ltd
Bridge Cottage,
Wilcot,
Pewsey,
Wilts. SN9 5NN

Mobile 07780 730 782

chris@dahu.co.uk

© Dahu Ltd 2026 .

All Rights Reserved. No part of this document may be photocopied, reproduced, stored in a retrieval system, or transmitted, in any form or by any means whether, electronic, mechanical, or otherwise without the prior written permission of Dahu LTD.

No warranty of accuracy is given concerning the contents of the information contained in this publication. To the extent permitted by law no liability (including liability to any person by reason of negligence) will be accepted by Dahu LTD, its subsidiaries or employees for any direct or indirect loss or damage caused by omissions from or inaccuracies in this document.

Dahu LTD reserves the right to change details in this publication without notice.

Windows is a trademark and Microsoft, MS-DOS, and Windows NT are registered trademarks of Microsoft Corporation. Other product and company names herein may be the trademarks of their respective owners.

Company Name: Dahu Ltd

Registered Number: 390384

Street Address: Bridge Cottage, Wilcot, Pewsey, Wiltshire SN9 5NN

Postal Address: Bridge Cottage, Wilcot, Pewsey, Wiltshire SN9 5NN

Contact Telephone: +44 (0)7780 730 782

Web Address: <http://www.dahu.co.uk>

Email Address: info@dahu.co.uk

Table of Contents

1. Introduction 5

2. Scope and Non-Goals 5

Architecture Overview 6

Data Source and Export 6

Network Storage as the Integration Layer 7

DahuVerityBuilder Service 7

DahuVerityIndexer Service 8

Verity Indexing Engine 8

Operational Characteristics 8

3. Directory Structure and File Flow 9

4. Service installation 9

5. Dahu Verity Builder Configuration 10

CSVDirectory 10

processedDirectory 11

quarantineDirectory 12

outputDirectory 12

pollInterval 14

deleteProcessedFiles 14

logDirectory 15

LogLevel 15

collectionPath 16

lettercodeMappingFilePath 17

defaultCollection 18

user.dir (implicit) 18

Configuration Completeness Statement 19

6. Dahu Verity Indexer Configuration 20

blkDirectory 20

processedDirectory 21

quarantineDirectory 21

mkvdkDir 22

mkvdkTimeoutSeconds 23

PollInterval 23

moveRemainingFiles 24
zipDirectory 24
zipFiles 25
logDirectory 26
LogLevel 27
user.dir (implicit) 27
Configuration Completeness Statement 27

7. Logging and Log Interpretation 29

8. Troubleshooting 29

Builder Running but Doing Nothing 29
Files Quarantined Unexpectedly 29
Indexer Reprocessing Same Editset 29
mkvdk Failures 30

9. DahuVerityBuilder Runbook – warnings & errors 31

Logging / service bootstrap 31
CSV title / content sanitation 31
File moves and cleanup (processed / quarantine housekeeping) 31
Watched directory polling and resilience 33
Directory creation at startup / ongoing 34
Lettercode mapping file (LetterCodeTranslator) 34
Editset file loading / parsing 37
Output generation (SGML / BLK / .index) 39
Reference building (data quality / malformed fields) 40

DahuVerityIndexer Runbook – warnings & errors 42

Logging / service bootstrap 42
Watched directory polling / resilience 42
Control file claiming / coordination 43
Control file parsing and validation 44
mkvdk execution lifecycle 45
File movement and cleanup 47

Escalation and Support 50

1. Introduction

Dahu Verity Builder and Dahu Verity Indexer together replace the Verity-related responsibilities of the legacy Releaser.exe component of the Procat editorial system.

Releaser.exe combined editorial release logic, database access, file generation, and Verity indexing in a single executable. It relied heavily on DCOM and a substantial body of legacy editorial code which can no longer be built or supported. DCOM itself is no longer a viable runtime technology.

As Releaser.exe degraded, fallback behaviour caused Verity indexing to run on the SQL Server rather than a dedicated indexing server. This introduced unacceptable operational risk, resource contention, and opaque failure modes.

The Builder and Indexer services remove these risks by separating responsibilities, using explicit file-based contracts, and enabling indexing to run on dedicated infrastructure. The result is a predictable, observable, and supportable indexing pipeline.

2. Scope and Non-Goals

SQL Server now owns all editorial decision-making and release logic. The Builder and Indexer operate only on data explicitly released by SQL in the form of CSV files. As part of the refactoring project, a new set of Stored Procedures were produced by TNA and Dahu to replicate the data gathering aspects of releaser.exe. These Stored Procedures export the edit set data in a set of CSV files that the Dahu Verity Builder can read to produce the indexable files. The Stored Procedures are not documented in this document.

These services:

- Do generate Verity-compatible BLK and SGML files
- Do execute Verity indexing via mkvdk

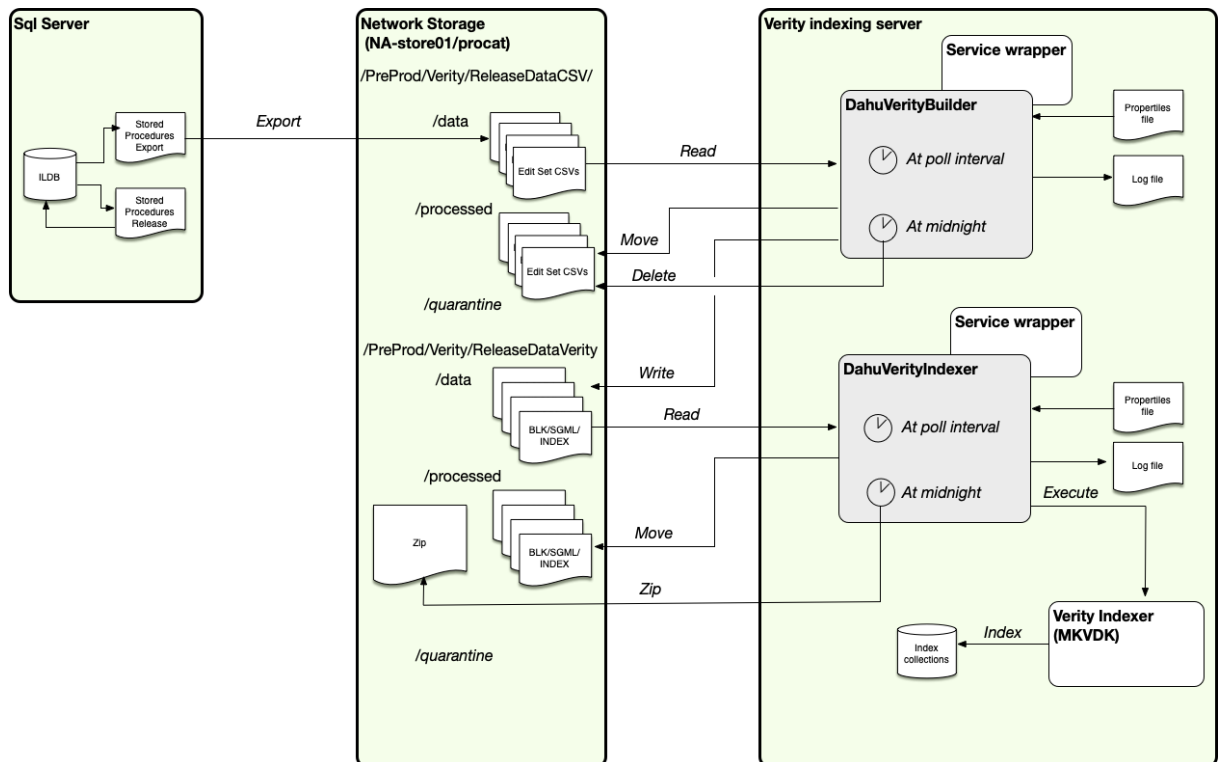
They do not:

- Contain editorial business rules
- Access the database directly
- Attempt to correct malformed upstream data

Architecture Overview

The Verity release and indexing architecture is designed as a decoupled, file-based pipeline that cleanly separates database activity, file generation, and indexing execution. This approach replaces the monolithic Releaser.exe with two long-running Java services — DahuVerityBuilder and DahuVerityIndexer — coordinated via shared network storage and simple, auditable file hand-offs.

The architecture deliberately avoids tight coupling between components, allowing each stage to operate independently, recover safely from failure, and be observed and supported operationally.



Data Source and Export

The source of all released content is the **ILDB** database on SQL Server.

Two sets of stored procedures are involved:

Export stored procedures generate editset data as CSV files.

Release stored procedures manage release semantics and status within the database.

CSV files are written by SQL Server to a shared **network storage location**:

```
Na-store-1/procat/PreProd/Verity/ReleaseDataCSV/data
```

These CSV files represent discrete, immutable release units and form the sole input to the downstream processing pipeline.

Network Storage as the Integration Layer

Shared network storage (NA-store01/procat) acts as the **integration boundary** between systems. It provides:

- Loose coupling between SQL Server, Builder, and Indexer
- Clear visibility of work in progress
- Simple operational recovery (files can be inspected, moved, or replayed)

Distinct directory trees are used for each stage, with explicit lifecycle transitions:

CSV input area

```
/PreProd/Verity/ReleaseDataCSV/  
├─ data      (new CSVs from SQL Server)  
├─ processed (successfully handled CSVs)  
└─ quarantine (invalid or rejected CSVs)
```

Verity file output area

```
/PreProd/Verity/ReleaseDataVerity/  
├─ data      (BLK, SGML, and .index files)  
├─ processed (successfully indexed files)  
└─ quarantine (failed or invalid index jobs)
```

DahuVerityBuilder Service

DahuVerityBuilder runs as a long-lived Windows service under a service wrapper.

Its responsibilities are strictly limited to **file creation and validation**:

- Periodically polls the CSV `data` directory
- Verifies file stability and structural validity
- Reads and processes editset CSV files
- Generates:
 - Verity **SGML**
 - Verity **BLK**
 - A small **.index control file** for the Indexer

All outputs are written to:

```
/PreProd/Verity/ReleaseDataVerity/data
```

On completion:

- Successfully processed CSV files are moved to `/processed`
- Invalid or problematic CSVs are moved to `/quarantine`
- No indexing is performed by the Builder

The Builder also performs scheduled housekeeping tasks (such as log rollover) independently of its polling cycle.

DahuVerityIndexer Service

DahuVerityIndexer is a separate Windows service, also managed via a service wrapper.

Its responsibilities are focused solely on **executing Verity indexing**:

- Periodically polls the Verity `data` directory for `.index` files
- Reads the BLK/SGML references produced by the Builder
- Invokes the Verity indexing engine (MKVDK)
- Writes index data into the appropriate Verity collections

On successful indexing:

- Input BLK/SGML/INDEX files are moved to `/processed`
- Optionally zipped and stored in `/zip` for audit or rollback

On failure:

- Files are moved to `/quarantine`
- Errors are logged with sufficient detail for diagnosis

The Indexer does not modify or regenerate content; it strictly consumes Builder output.

Verity Indexing Engine

The Verity indexing engine (MKVDK) runs locally on the indexing server and is invoked as an external process by DahuVerityIndexer.

It reads SGML and BLK files from shared storage and writes directly to the configured **Verity index collections**. This preserves Verity's expected operating model while isolating it from database and file-generation concerns.

Operational Characteristics

This architecture provides several important operational benefits:

- **Deterministic processing**: each stage has a single, well-defined responsibility
- **Failure isolation**: CSV handling, file generation, and indexing failures are independent
- **Recoverability**: quarantined files can be inspected and replayed safely
- **Auditability**: all intermediate artifacts are visible and persistent
- **Deployment flexibility**: Builder and Indexer can be restarted, upgraded, or tuned independently

Overall, the design prioritises **predictability, traceability, and operational safety** over tight coupling or hidden state, making it suitable for long-term unattended operation in a production archival environment.

3. Directory Structure and File Flow

The system relies heavily on filesystem state. Understanding directory contents is essential for operational support.

Example directory layout (non-production):

```
Network\na-store01procat\PreProd\Verity\ReleaseDataCSV\
```

```
data\      ← CSV files emitted by SQL
```

```
processed\ ← Successfully processed CSVs
```

```
quarantine\ ← Invalid or unsafe CSVs
```

```
Network\na-store01\procat\PreProd\Verity\ReleaseDataVerity\
```

```
data\      ← BLK, SGML, and .index files
```

```
processed\ ← Indexed BLK, SGML, and .index files
```

```
quarantine\ ← BLK,SGML & .index files that failed successful indexing
```

Files move strictly top-to-bottom through this structure. Manual movement backwards constitutes reprocessing and must be deliberate. Typically, DahuVerityBuilder will be configured to write its output file (BLK, SGML and .index files) to the watched data directory of the DahuVerityIndexer service.

4. Service installation

Both DahuVerityBuilder and DahuVerityIndexer are java programs designed to run as a service on Windows. To enable service integration, we use a service wrapper called NSSM.EXE. The executable can be found on any of the Verity indexing servers (Prod, Pre-Prod, Test) at:

```
D:\Program Files(x86)\Dahu\service\nssm-2.24\nssm-2.24\win64\nssm.exe
```

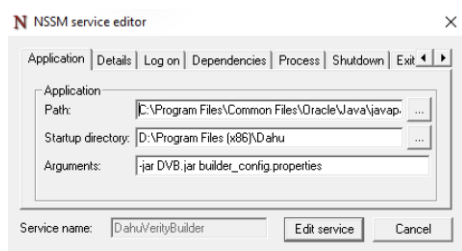
To install either of the services for the first time, from a command window, run:

```
nssm.exe install <<name of the service>>
```

so for DahuVerityBuilder: `nssm.exe install DahuVerityBuilder`

and for DahuVerityIndexer: `nssm.exe install DahuVerityIndexer`

When the nssm.exe window is presented, add the following configuration values:



Path: This is the path to the java.exe installed on the server, normally:

```
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe
```

Startup directory: The default location for the tools. DVB.JAR and DVI.JAR will be in this directory as will the properties file for the service.

Usually, it will be `C:\Program Files (x86)\Dahu`

Arguments: These are the arguments to the java.exe to start the service. For DahuVerityBuilder the arguments will be

```
-jar DVB.jar builder_config.properties
```

and for DahuVerityIndexer

```
-jar DVI.jar indexer_config.properties
```

Should you need to edit the service configuration, you can run the nssm.exe tool in edit mode using the command `nssm.exe edit <<service name>>`

5. Dahu Verity Builder Configuration

This section describes **every configurable parameter** supported by the Dahu Verity Builder service.

Each parameter is documented in terms of:

- Purpose and functionality
- Example value
- Observable logging behaviour
- Operational guidance

The configuration is typically supplied as a Java `.properties` file, provided as a parameter to the service and read at service startup. Here is an example properties file for the Builder process:

Example_DahuBuilder.properties

```
CSVDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataCSV/data
processedDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataCSV/processed
quarantineDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataCSV/quarantine
outputDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataVerity/data
LogLevel=FINEST
pollInterval=3
deleteProcessedFiles=true
```

CSVDirectory

Purpose

Specifies the directory monitored for incoming CSV files generated by SQL Server release stored procedures.

Functionality

- The Builder polls this directory at the configured `pollInterval`
- Files found here are candidates for processing once they are deemed stable
- Only files in this directory are ever considered as new work

Example

```
CSVDirectory=/data/ReleaseDataCSV/data
```

Logging

On startup:

```
INFO Started watching /data/ReleaseDataCSV/data directory.
```

During idle periods:

```
FINEST No files found in the watched directory. Sleeping until next interval.
```

When work is detected:

```
FINEST Files found in the watched directory. Processing N files.
```

Operational Guidance

- This directory **must only** be written to by SQL Server processes
- Manual placement of files is strongly discouraged
- Presence of files here indicates upstream release activity
- Persistent emptiness usually indicates an upstream (SQL) issue, not a Builder fault

processedDirectory

Purpose

Defines where successfully processed CSV files are moved after completion.

Functionality

- After a CSV file has been fully processed and all output artefacts written, it is moved here
- Prevents reprocessing of the same editset
- Acts as an audit trail of completed work

Example

```
processedDirectory=/data/ReleaseDataCSV/processed
```

Logging

```
INFO Moved processed editSet file '/data/.../editset.csv' to directory  
/data/ReleaseDataCSV/processed
```

Operational Guidance

- Files in this directory represent **successful completion**
- Retain for audit and troubleshooting
- Moving files back to `CSVDirectory` will cause reprocessing and should only be done deliberately
- Absence of expected files here suggests processing did not complete

quarantineDirectory

Purpose

Isolates CSV files that cannot be processed safely.

Functionality

Files are quarantined when Builder detects conditions that make processing unsafe or non-deterministic, including (but not limited to):

- CSV parsing errors
- Invalid or unexpected filename format
- Missing mandatory data
- Inconsistent editset metadata
- Conditions where deterministic output guarantees cannot be met

Quarantined files are **never retried automatically**.

Example

```
quarantineDirectory=/data/ReleaseDataCSV/quarantine
```

Logging

Typical entries include:

```
WARNING Failed to process editSet file ... moving to quarantine
```

Operational Guidance

- Quarantine is **intentional and protective**, not an error state
- Files here must be reviewed manually
- Root cause must be fixed upstream (SQL or data generation)
- Reintroducing files without fixing the cause will result in repeated quarantine
- Quarantine growth is a strong signal of upstream data quality issues

outputDirectory

Purpose

Specifies the root directory where Verity BLK, SGML, and `.index` files are written.

Functionality

Builder creates deterministic filenames and directory structures under this root

Output here forms the input contract for the Indexer service

Example

```
outputDirectory=/data/ReleaseDataVerity/data
```

Logging

```
INFO Writing index file /data/ReleaseDataVerity/data/<filename>.index
```

```
INFO Files written for editset file '<csv-file>'
```

Operational Guidance

- Must be writable by the Builder service account
- Must be readable by the Indexer service
- Disk space exhaustion here will halt the pipeline
- Manual modification of files in this directory is strongly discouraged

pollInterval

Purpose

Controls how frequently the Builder scans the CSVDirectory for new files.

Functionality

- Determines the sleep interval between directory scans
- Expressed in seconds

Example

```
pollInterval=3
```

Logging

Idle behaviour is visible as:

```
FINEST No files found in the watched directory. Sleeping until next interval.
```

Operational Guidance

- Lower values reduce latency but increase filesystem activity
- Higher values reduce load but delay processing
- Values under ~2 seconds rarely provide meaningful benefit
- Changes do not require code changes but do require service restart

deleteProcessedFiles

Purpose

Controls whether processed CSV files are deleted rather than moved. Files are deleted when logs rollover which occurs at midnight.

Functionality

- When `true`, processed CSVs are removed instead of being retained
- When `false`, CSVs are retained in `processedDirectory`

Example

```
deleteProcessedFiles=true
```

Logging

When enabled, absence of logs is expected.

Operational Guidance

- **Strongly recommended to leave as `true` in production**
- Deletion removes valuable audit and troubleshooting evidence
- Only consider enabling in controlled, low-risk environments
- Does not affect quarantine behaviour

logDirectory

Purpose

Specifies the directory where the Builder writes its log files.

Functionality

- Controls the physical location of Builder log output
- Log files are created and rolled according to the Builder's rollover logic (date-based)
- This is separate from the CSV/Verity data directories; it should be treated as operational telemetry, not pipeline data

Example

```
logDirectory=/var/log/dahu/verity-builder
```

Logging

Typical early log entries will confirm log configuration, for example:

- logger initialisation / configuration output (and the actual resolved log file name)
- ongoing operational state, e.g. directory polling and processing lifecycle messages

Operational Guidance

- Ensure the service account has **create/write** permission in this directory
- Ensure log retention/rotation aligns with your operational policy (the Builder rolls daily; retention is typically handled by ops policy, backups, or housekeeping)
- If logs stop updating:
 - check disk space
 - check permissions
 - check whether log rollover has failed due to path issues

LogLevel

Purpose

Controls logging verbosity. Changing the logLevel in the properties file requires a restart of the service for the change to take effect.

Functionality

- Passed directly to Java logging configuration
- Controls volume and granularity of log output

Example

```
LogLevel=FINEST
```

Observed Levels

- SEVERE – fatal errors
- WARNING – recoverable or fallback conditions

- `INFO` – normal lifecycle events
- `FINE` / `FINEST` – detailed operational tracing

Operational Guidance

- `INFO` is appropriate for steady-state production
- `FINEST` is suitable for troubleshooting and validation
- High verbosity increases log volume but improves diagnosability
- Log level changes require service restart

collectionPath

Purpose

Defines the base path used to form Verity collection locations referenced in the generated `.index` files and/or file output structure.

Functionality

Provides the “root” under which the Builder expects Verity collections to exist

Used together with letter code → collection mapping (see `lettercodeMappingFilePath`) to resolve the final collection path used by indexing

Allows collections to be relocated without altering upstream SQL behaviour or Builder code

Example

```
collectionPath=/verity/collections
```

Logging

You should expect log entries that reflect resolved collection selection, for example:

- explicit mapping/lookup messages (where enabled at higher verbosity)
- warnings when mapping is missing and a fallback is applied (see `defaultCollection`)

Operational Guidance

- Treat this as a **high-impact setting**: a wrong value can cause the Indexer to run `mkvdk` against the wrong place or fail to find the collection entirely
- If indexing appears to “succeed” but content is not where you expect, this is one of the first parameters to verify
- Prefer absolute paths on production servers to avoid dependency on process working directory (`user.dir` / NSSM Startup Directory)

lettercodeMappingFilePath

Purpose

Defines the path to the letter-code-to-collection mapping file used to determine which Verity collection a given editset targets.

Functionality

- The Builder loads this mapping file to translate a “letter code” (from the input / filename conventions) into a collection name/path
- The mapping is intended to be:
 - safe to reload when updated
 - tolerant of comments/blank lines
 - robust against malformed lines
 - capable of **keeping the last-known-good mapping** if a reload fails
- If a letter code cannot be mapped, the Builder falls back to `defaultCollection`

Example

```
lettercodeMappingFilePath=/etc/dahu/lettercode-mapping.txt
```

Logging

Typical log patterns:

- On successful load or reload:
- an INFO-level message indicating the file was loaded (often including modified time)

On parse issues:

- warnings about malformed lines being ignored

On missing mapping:

- a warning indicating there was no mapping for a given letter code and a fallback will be used

Operational Guidance

- This file is operational configuration: control it like you would any other config artefact (change control, backup, audit trail)
- If you see repeated “no mapping found” warnings:
- check that the letter code is expected
- update the mapping file (preferred)
- avoid “fixing” this by changing `defaultCollection` unless the fallback is genuinely correct

- If reload fails (syntax/IO), Builder should continue using last-known-good mapping; do **not** restart the service as a first reaction — fix the mapping file and let it reload cleanly

defaultCollection

Purpose

Specifies the default Verity collection to use when a letter code cannot be mapped.

Functionality

- Provides a controlled fallback so Builder can continue generating outputs even when mapping coverage is incomplete
- Prevents the pipeline from failing hard on a single unmapped letter code
- Used only when `lettercodeMappingFilePath` lookup fails for a particular code (or if the mapping file is missing/unusable and no last-known-good mapping is available)

Example

```
defaultCollection=Advanced/adv1
```

Logging

Typical pattern when invoked:

- a warning indicating missing mapping
- a note that fallback/default collection is being used (verbosity-dependent)

Operational Guidance

- This should be set to a **safe, conservative default** agreed with the Verity/indexing design
- If you find this being used frequently, treat it as a configuration/data quality issue:
 - update the mapping file
 - or correct upstream letter codes

Over-reliance on `defaultCollection` can lead to indexing in the wrong place, which is operationally worse than a controlled failure; monitor warnings and address root cause

user.dir (implicit)

Purpose

Not configurable directly, but logged for diagnostic purposes.

Functionality

- Reflects the working directory of the JVM process
- Useful when diagnosing relative path issues

Logging

```
FINE user.dir = /path/to/working/dir
```

Operational Guidance

- Ensure NSSM startup directory is explicitly set
- Avoid reliance on relative paths in configuration

Configuration Completeness Statement

The parameters documented above represent **the complete set of configurable properties** supported by the Dahu Verity Builder service as implemented in the current codebase and exercised in runtime logs.

No additional hidden or undocumented configuration options exist.

6. Dahu Verity Indexer Configuration

This section describes **every configurable parameter** supported by the Dahu Verity Indexer service. Each parameter is documented in terms of:

- Purpose and functionality
- Example value
- Observable logging behaviour
- Operational guidance

The configuration is typically supplied as a Java `.properties` file, provided as a parameter to the service and read at service startup. Here is an example properties file for the Builder process:

Example_DahuIndexer.properties

```
blkDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataVerity/data
processedDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataVerity/processed
quarantineDirectory=/Users/chrisbartlett/Development/TNA/ReleaseDataVerity/quarantine
LogLevel=FINEST
mkvdkDir=/Users/chrisbartlett/Development/TNA
collectionMappingPath=/Users/chrisbartlett/Development/TNA/lettercodeMapping.csv
PollInterval=3
oveRemainingFiles=no
zipFiles=no
mkvdkTimeoutSeconds=3600
```

blkDirectory

Purpose

Specifies the directory containing Verity BLK, SGML, and `.index` files produced by the Builder that are to be indexed by Verity.

Functionality

- This directory is monitored for `.index` control files
- Each `.index` file represents a complete, immutable instruction to perform an indexing operation
- Presence of files here indicates pending indexing work

Example

```
blkDirectory=/data/ReleaseDataVerity/data
```

Logging

On startup:

```
INFO Started watching /data/ReleaseDataVerity/data directory.
```

When work is detected:

Operational Guidance

- This directory forms the **contract boundary** between Builder and Indexer
- Files appearing here should never be modified manually
- Growth of files here indicates indexing backlog
- Absence of files here while Builder is active usually indicates an upstream issue

processedDirectory

Purpose

Defines where BLK, SGML, and `.index` files are moved after indexing has been attempted.

Functionality

- After indexing completes (successfully or unsuccessfully), files are moved here
- Prevents re-execution of the same indexing instruction
- Acts as an audit and replay source

Example

```
processedDirectory=/data/ReleaseDataVerity/processed
```

Logging

```
INFO Moved processed index file '/data/.../<filename>.index' to directory  
/data/ReleaseDataVerity/processed
```

Operational Guidance

- Files here indicate indexing has already been attempted
- Retain for audit, diagnostics, and controlled reprocessing
- Moving files back to `blkDirectory` will cause re-indexing and must be deliberate
- Warnings about files “not present” during move typically indicate reprocessing and are usually benign

quarantineDirectory

Purpose

Isolates indexing inputs that cannot be processed safely or consistently.

Functionality

Files may be quarantined if:

- Required BLK or SGML files are missing
- Index instructions are malformed
- Index paths resolve inconsistently
- Preconditions for deterministic indexing are not met

Quarantined files are **not retried automatically**.

Example

```
quarantineDirectory=/data/ReleaseDataVerity/quarantine
```

Logging

Typical entries include:

```
WARNING Failed to process index file ... moving to quarantine
```

Operational Guidance

- Quarantine protects the system from repeated failed indexing attempts
- Files here require **manual investigation**
- Root cause is often upstream (Builder output or configuration)
- Reintroduce files only after correcting the cause
- Persistent quarantine indicates systemic configuration or data issues

mkvdkDir

Purpose

Specifies the directory containing the Verity `mkvdk` executable.

Functionality

- Indexer invokes `mkvdk` directly from this location
- Used for insert, update, and delete operations
- Invocation parameters are derived from `.index` control files

Example

```
mkvdkDir=/opt/verity/bin
```

Logging

```
INFO Executing mkvdk from /opt/verity/bin
```

Failures typically surface as:

```
SEVERE mkvdk execution failed
```

Operational Guidance

- Path must be valid and executable by the service account
- Incorrect paths result in immediate indexing failure
- Changes require service restart
- Ensure `mkvdk` version matches Verity index format in use

mkvdkTimeoutSeconds

Purpose

Defines the maximum allowed runtime for a single mkvdk invocation.

Functionality

- Indexing operations exceeding this duration are terminated
- Prevents hung or runaway indexing processes
- Expressed in seconds

Example

```
mkvdkTimeoutSeconds=3600
```

Logging

Timeout events typically appear as:

```
WARNING mkvdk execution exceeded timeout of 3600 seconds
```

Operational Guidance

- Timeouts are serious and should be investigated
- Common causes include:
 - Corrupt BLK files
 - Disk I/O contention
 - Index corruption
- Review mkvdk log output immediately
- Increasing this value should be a last resort

PollInterval

Purpose

Controls how frequently the Indexer scans the `blkDirectory` for `.index` files.

Functionality

- Determines idle sleep duration between scans
- Expressed in seconds

Example

```
PollInterval=3
```

Logging

Idle behaviour appears as:

```
FINEST No index files found. Sleeping until next interval.
```

Operational Guidance

- Lower values reduce indexing latency

- Higher values reduce filesystem activity
- Extremely low values provide little benefit
- Changes require service restart

moveRemainingFiles

Purpose

Controls whether unprocessed files are moved to processed after indexing completes. This will include any files of any type such that the directory remains clean.

Functionality

- When enabled, all files in `blkDirectory` are moved to `processedDirectory`
- When disabled, any unprocessable files will remain in `blkDirectory`

Example

```
moveRemainingFiles=no
```

Logging

When enabled:

```
INFO Moving remaining BLK and SGML files to processed directory
```

Operational Guidance

- Leaving related files in place can be useful during validation
- Moving all artefacts simplifies housekeeping
- Choice should align with retention and audit policy
- Changing behaviour mid-stream can complicate recovery

zipDirectory

Purpose

Specifies the directory where zipped indexing artefacts are written when zipping of processed files is enabled.

Functionality

- Used only when `zipFiles=yes`
- After an indexing operation completes and files are moved to `processedDirectory`, the Indexer creates a ZIP archive containing:
 - the `.index` file
 - associated BLK and SGML files (subject to `moveRemainingFiles`)
- The resulting ZIP file is written to `zipDirectory`
- Original unzipped files may then be deleted or retained depending on Indexer behaviour and configuration

Example

```
zipDirectory=/data/ReleaseDataVerity/zipped
```

Logging

Typical log patterns include:

When zipping is initiated:

```
INFO Zipping processed index artefacts for <index-name>
```

On successful archive creation:

```
INFO Created zip archive /data/ReleaseDataVerity/zipped/<index-name>.zip
```

On failure:

```
WARNING Failed to zip processed files for <index-name>
```

Operational Guidance

- This directory must be writable by the Indexer service account
- Disk usage here can grow quickly if retention is not managed — zipped files tend to accumulate silently
- Zipped artefacts are **not intended for routine inspection**:
 - enable zipping primarily for archival or transfer purposes
 - leave `zipFiles=no` for normal operations
- If zipping fails:
 - indexing itself may already have completed successfully
 - treat zipping failures as secondary unless storage/retention is a contractual requirement
- If `zipFiles=yes` and `zipDirectory` is missing or misconfigured:
 - expect repeated warnings
 - processed artefacts may remain unzipped in `processedDirectory`

zipFiles

Purpose

Controls whether processed indexing artefacts are zipped after completion. Zipping occurs at midnight rollover. The log therefore contain all of the previous day's work.

Functionality

- When enabled, processed files are compressed
- Reduces disk usage at the cost of accessibility

Example

```
zipFiles=no
```

Logging

When enabled:

```
INFO Zipping processed index artefacts
```

Operational Guidance

- Mirrors existing procat behaviour
- Generally unnecessary unless disk space is constrained
- Zipped artefacts complicate manual inspection
- Recommend leaving disabled unless required

logDirectory

Purpose

Specifies the directory where the Indexer writes its log files.

Functionality

- Controls the physical location of Indexer log output
- Log files are created and rolled according to the Indexer's rollover logic (date-based)
- This is separate from the BLK directories; it should be treated as operational telemetry, not pipeline data

Example

```
logDirectory=/var/log/dahu/verity-indexer
```

Logging

Typical early log entries will confirm log configuration, for example:

- logger initialisation / configuration output (and the actual resolved log file name)
- ongoing operational state, e.g. directory polling and processing lifecycle messages

Operational Guidance

- Ensure the service account has **create/write** permission in this directory
- Ensure log retention/rotation aligns with your operational policy (the Indexer rolls daily; retention is typically handled by ops policy, backups, or housekeeping)
- If logs stop updating:
 - check disk space
 - check permissions
 - check whether log rollover has failed due to path issues

LogLevel

Purpose

Controls verbosity of Indexer logging.

Functionality

- Passed directly to Java logging framework
- Controls volume and diagnostic detail

Example

```
LogLevel=FINEST
```

Observed Levels

- SEVERE – fatal indexing failures
- WARNING – recoverable or degraded behaviour
- INFO – lifecycle and execution events
- FINE / FINEST – detailed execution tracing

Operational Guidance

- INFO is suitable for steady-state production
- FINEST should be used during investigation
- High verbosity increases log volume significantly
- Changes require service restart

user.dir (implicit)

Purpose

Not configurable directly, but logged for diagnostic clarity.

Functionality

- Reflects the JVM working directory
- Useful when diagnosing relative path issues or NSSM misconfiguration

Logging

```
FINE user.dir = /path/to/indexer/working/dir
```

Operational Guidance

- Ensure NSSM startup directory is explicitly configured
- Avoid reliance on relative paths in configuration

Configuration Completeness Statement

The parameters documented above represent **the complete set of configurable properties** supported by the Dahu Verity Indexer service as implemented in the current codebase and exercised via supplied configuration and runtime logs.

No undocumented or hidden configuration options exist.

7. Logging and Log Interpretation

Both services use structured logging with timestamps and severity levels. Logs are written to the working directory -

Common log patterns:

- INFO Started watching directory – service healthy
- FINEST No files found – idle state
- WARNING no collection found – fallback mapping in use
- WARNING File to move is not present – duplicate or re-run scenario

8. Troubleshooting

Builder Running but Doing Nothing

Symptoms:

- Repeated 'No files found' messages

Checks:

- Confirm SQL is emitting CSV files
- Verify CSVDirectory path
- Check permissions

Files Quarantined Unexpectedly

Symptoms:

- Files appear in quarantineDirectory

Actions:

- Inspect Builder log around quarantine event
- Validate CSV format and filename
- Check collection mapping coverage

Indexer Reprocessing Same Editset

Symptoms:

- Repeated indexing of same editset
- Warnings about missing files during move

Cause:

- Manual reintroduction of .index files

Resolution:

- Confirm desired reprocessing
- Otherwise, clean up control directories

mkvdk Failures

Symptoms:

- Indexing command fails or times out

Actions:

- Inspect mkvdk .log files
- Verify collection paths
- Verify disk space and permissions

9. DahuVerityBuilder Runbook – warnings & errors

Logging / service bootstrap

SEVERE: Failed to create file logger; continuing with console-only logging.

What it means

Builder tried to create the file logger and failed, so it will log to console only.

What to do

- Check `cfg.logDirectory` exists and is writable by the service account.
- Check the log path isn't on a disconnected share.
- Check for Windows "file in use" / permission issues.

You'll still get logs in the console (NSSM/stdout etc.), but you'll lose file-based operational history until fixed.

Escalate if

This repeats every start and you rely on file logs for audit / troubleshooting.

CSV title / content sanitation

WARNING: Problem while truncating title. <message>

What it means

A defensive truncation routine failed; Builder will fall back to an empty title for that record.

Likely causes

- Unexpected nulls, very short strings, odd punctuation, unexpected encoding.
- Malformed input data (e.g., control characters).

What to do

- Usually safe to ignore unless you see it frequently.
- If frequent: inspect the input data for the record(s) being processed; look for fields containing odd characters / invalid markup.
- Consider adding extra defensive checks upstream (SQL export sanitisation).

File moves and cleanup (processed / quarantine housekeeping)

WARNING: Error while trying to move files: <message>

What it means

Builder attempted to move a CSV into another directory (processed/quarantine) and the move failed.

Likely causes

- Permissions on `processedDirectory` or `quarantineDirectory`
- Destination file locked/in use
- Network share transient failure
- Path invalid (especially if config contains backslash escapes in `.properties`)

What to do

- Check directory exists and is writable.
 - If on UNC share, confirm connectivity and credentials.
 - Check whether an AV product is locking CSVs.
 - Confirm you're not feeding paths like `C:\tmp\out` in `.properties` without escaping (`\t` becomes `tab`).
-

WARNING: File to move is not present: <fileName>

What it means

Builder expected a file to exist when trying to move it, but it wasn't there.

Likely causes

- Another process already moved/deleted it
- Race: CSV was written then removed by export process / cleanup job
- Wrong watched directory

What to do

- Confirm the SQL export process is writing to the same `cfg.csvDirectory` Builder watches.
 - Check for other cleanup jobs touching the directory.
 - If frequent, add "handoff" convention (e.g., write to `.tmp` then rename to `.csv` when complete).
-

WARNING: Failed to remove <filePath>

WARNING: Failed to remove <filePath> file not found

WARNING: Failed to remove file <filePath>. <message>

What it means

Builder is tidying processed files (when `deleteProcessedFiles` is enabled) and couldn't delete something.

Likely causes

- Permissions / ownership

- File locked (indexer, antivirus, user)
- Path points somewhere unexpected

What to do

- Check the processed directory is correct and is a “Builder-owned” location.
- On Windows, look for locks using Sysinternals `handle.exe` or similar.
- If this is a network share, check share permissions + connectivity.

Watched directory polling and resilience

WARNING: Cannot read watched directory: <csvDirectory> (will retry; throttled...) Cause: <Exception>: <message>

What it means

Builder couldn't list the watched directory this tick. It will retry next polling interval, and it will only log this warning at most once per throttle window.

Likely causes

- Directory does not exist
- Permissions
- Network share temporarily unavailable
- Path mis-specified in config (escape sequences again)

What to do

- Verify directory exists and is readable by the service account.
- If UNC share, validate the service identity has access.
- Confirm `csvDirectory` config uses safe path format:
- Preferred: `//server/share/path` or `C:/path/...`
- If using backslashes in `.properties`, they must be doubled.

SEVERE: Unhandled error in watcher loop

What it means

Something unexpected escaped the normal per-file error handling in the main polling loop.

What to do

- Treat as “investigate immediately”.
- Look at the exception stack trace: it should identify the failing subsystem (directory read, mapping refresh, file parse, output writing, etc.).
- If it repeats every tick, Builder may be effectively stuck until restart or configuration fix.

Directory creation at startup / ongoing

WARNING: Unable to create/watch CSVDirectory: <csvDirectory> (will keep trying during polling)

What it means

Builder couldn't create the watched directory. It won't exit; it will keep trying.

What to do

- Create the directory manually, verify permissions.
- If it's a UNC share, ensure the share is reachable at service start time.
- If this is intentional (directory managed elsewhere), you still need it to exist before Builder can do useful work.

WARNING: Unable to create outputDirectory: <outputDirectory>

WARNING: Unable to create processedDirectory: <processedDirectory>

WARNING: Unable to create quarantineDirectory: <quarantineDirectory>

WARNING: Unable to create logDirectory: <logDirectory>

What it means

Builder couldn't create one of its required working directories.

Likely causes

- Permissions / ACLs
- Invalid path string (again: `.properties` escaping)
- Parent path missing on a network share
- Disk full / read-only volume

What to do

- Ensure each directory exists and is writable.
- Prefer forward slashes in config:
 - `outputDirectory=//na-stor01/procat/verity/out`
 - `logDirectory=C:/procat/logs`

Confirm the service account has rights.

Lettercode mapping file (LetterCodeTranslator)

WARNING: Lettercode mapping file not found: <path> (will use defaultCollection=<default>)

What it means

Mapping file is missing and there is **no previously loaded mapping**, so Builder will fall back to defaults.

Operational impact

- Collections for ADV-style indexing may default to `Advanced/<defaultCollection>`.

What to do

- Put the mapping file in place at the configured location.
- Confirm path is correct and readable.
- Confirm file name case/extension if on a case-sensitive share.

WARNING: Lettercode mapping file not found: <path> (keeping last-known-good mappings)

What it means

Mapping file is missing *now*, but Builder previously loaded it successfully and will keep using the cached mapping.

What to do

- Restore the mapping file (someone moved/renamed it).
- Treat as “needs fixing soon”, but it is not immediately catastrophic.

SEVERE: Failed to load lettercode mapping file: <path> (no previous mapping available; will use defaults)

What it means

An `IOException` occurred reading the mapping file and there is no cached mapping.

Likely causes

- File unreadable (permissions)
- Locked file
- Network share glitch
- File path points somewhere invalid

What to do

- Fix connectivity/permissions.
- If on share: copy file locally and point config at local path to remove share dependency.
- Once fixed, Builder will load it on next tick (it calls refresh each tick).

WARNING: Failed to reload lettercode mapping file: <path> (keeping last-known-good mappings)

What it means

Reload failed, but Builder will continue using cached last-known-good mapping.

What to do

- Investigate why reload failed: partial write, file replaced while being read, share issues.
- Best practice: mapping file updates should be “atomic” (write new file then rename over old).

SEVERE: Unexpected error while parsing lettercode mapping file:
<path> (no previous mapping available; will use defaults)

WARNING: Unexpected error while parsing lettercode mapping file:
<path> (keeping last-known-good mappings)

What it means

The mapping file was found/read, but parsing hit an unexpected runtime error.

Likely causes

- Weird encoding / non-text content
- Extremely long lines / unexpected characters
- Some bug triggered by unusual format

What to do

- Open the mapping file and validate it's plain text, expected format.
- Ensure it's US-ASCII / UTF-8 without exotic characters.
- If this appeared right after an edit, roll back to a known-good copy.

WARNING: Malformed mapping line <n> (missing ':'): <line>

What it means

A non-empty, non-comment line didn't contain a : separator.

What to do

- Fix the line format in the mapping file.
- This is usually harmless unless it affects a lettercode you care about.

WARNING: Malformed mapping line <n> (empty collection/codes): <line>

What it means

Line had a colon, but either the collection name or code list was empty.

What to do

- Fix the line. Example valid line:
 - `adv1:WO,RAIL`
 - `adv2:FO,HO`

Editset file loading / parsing

WARNING: Failed when writing files for editset file '<editSetFile>'.

What it means

Builder parsed the editset but `writeFiles(...)` returned false, so it will move the CSV to quarantine.

What to do

- Look for earlier warnings/errors around output directory creation, filename generation, SGML/BLK writing, or index file writing.
- Quarantine is correct: it prevents repeated reprocessing of a broken batch.

SEVERE: Bad editset file, quarantining: <editSetFile>

What it means

The CSV couldn't be parsed into an `EditSetBatch` (illegal filename format, unreadable file, etc.). Builder quarantines it.

What to do

- Inspect the quarantined CSV:
 - Filename format (see below)
 - Encoding (expected UTF-8 read)
 - Header correctness
- If it's an export issue, fix SQL export and re-run.

SEVERE: Filename can't be parsed format doesn't look correct.

What it means

The CSV filename didn't split into at least 4 underscore-separated parts.

What to do

- Ensure exporter writes filenames in the expected pattern, e.g.:
- `<date>_<editSetId>_<something>_<mode>.csv`
- If you changed the naming scheme, Builder must be updated to match.

SEVERE: Can't find a processing mode in file name <fileName>

What it means

Mode indicator is absent from filename, so Builder doesn't know whether this is add/update/delete behaviour.

What to do

- Ensure exporter includes the mode part.
- Confirm the mode is in the correct underscore position.

SEVERE: Filename can't be parsed - looks like <parts[2]> is not a number.

What it means

A filename segment that Builder expects to be numeric wasn't numeric.

What to do

- Check filename template, and ensure the numeric field is numeric.
- Check for accidental extra underscores shifting positions.

WARNING: Row without ID; skipping

What it means

A CSV data row didn't contain an ID value.

What to do

- This is usually an export/data quality issue.
- Investigate the stored procedure export: ensure ID column is always present and populated.

WARNING: Row with non-numeric ID '<idRaw>'; skipping

What it means

The ID column was present but not numeric.

What to do

- Check export formatting (e.g., quoting, commas in wrong place).
- Validate the ID column isn't accidentally outputting header text mid-file due to concatenation bugs.

WARNING: Failed to merge additional data for ID=<id>

What it means

Builder saw a duplicate ID row (typically link/secondary data) and tried to merge it into an existing `EditSetRecord`, but merge failed.

Likely causes

- Secondary row missing expected fields
- Link data malformed
- Unexpected column layout

What to do

- Inspect the CSV rows for that ID.

- Confirm export produces consistent columns across all row “types”.

Output generation (SGML / BLK / .index)

SEVERE: Unable to create output directory: <outputDirectory>

What it means

Builder cannot create/access the output directory *at the point of writing* (this is inside `writeFiles(...)`).

What to do

- Fix ACLs / share connectivity.
- Confirm config path correctness.
- If UNC: make sure the service account can access the share non-interactively.

WARNING: no collection found in mapping for letterCode <letterCode>. <editSetId>

What it means

No mapping existed for that lettercode, so Builder will fall back to Advanced/<defaultCollection>.

What to do

- If this lettercode matters, add it to the mapping file.
- If it's expected (rare/unknown code), you can ignore.

WARNING: Failed to create output filename for editSet <editSetId>

What it means

Builder couldn't derive the output base filename for `.sgml/.blk`.

Likely causes

- Missing/invalid key fields used by the filename routine
- Unexpected collection name / lettercode empty
- Mode missing

What to do

- Check the input CSV contains the fields the filename routine expects.
- Check earlier parsing warnings (missing ID, empty lettercode, etc.).

WARNING: Failed while writing files for editset <editSetId> to file <blkPath> or <sgmlPath>

What it means

An `IOException` happened during SGML/BLK creation or while writing the `.index` entry.

What to do

- Check disk space and permissions on `outputDirectory`.
- Check whether those files are locked by something else.
- If output is on network storage, validate connectivity and latency.
- Confirm paths are correct and *safe for consumers* (especially on Windows: consider forward slashes in BLK `DOC_FN`, per our earlier discussion).

SEVERE: Unable to open/write log (control) file for editSet
<editSetId> : <indexPath>

What it means

Builder couldn't create or write the `.index` control file (the thing the Indexer consumes).

Impact

Indexer won't run for this editset even if SGML/BLK exist.

What to do

- Check directory permissions and disk space.
- Check whether `.index` file is locked (Indexer shouldn't lock it long-term, but AV might).
- Confirm `OUT_CHARSET` write is allowed (should be).

Reference building (data quality / malformed fields)

WARNING: trying to build reference but letterCode is empty or null.

WARNING: trying to build reference but classRef is empty or null.

WARNING: trying to build reference but pieceRef is empty or null.

What it means

Builder attempted to construct a catalogue reference for output, but required parts were missing.

What to do

- Treat as data quality / export completeness issue.
- Check the SQL export for null/empty values in those fields.
- If missing values are valid for certain catalogue levels, consider whether the checks should be conditional by level (but currently it logs and returns empty).

WARNING: Problem while building reference. <message>

What it means

An exception occurred in reference construction; Builder returns an empty reference string.

What to do

- Look at `<message>` and identify which part failed (often nulls or unexpected integer catalogue levels).

- Usually indicates either malformed CSV content or an unexpected catalogueLevel value.

Logging / service bootstrap

SEVERE: Failed to create file logger; continuing with console-only logging.

What it means

Indexer failed to initialise its file-based logger and will continue using console-only logging.

Likely causes

- `logDirectory` does not exist
- Permissions / ACL issue
- Log file already locked
- Network share unavailable

What to do

- Ensure `logDirectory` exists and is writable by the Indexer service account.
- Check for file locks or AV interference.
- Verify path format in config (avoid unescaped backslashes in `.properties`).

Operational impact

- Indexer continues running, but persistent logs are unavailable.

Watched directory polling / resilience

WARNING: Cannot read watched directory: `<blkDirectory>` (will retry; throttled). Cause: `<Exception>`: `<message>`

What it means

Indexer could not list the BLK/control-file directory during this polling cycle.

Likely causes

- Directory missing
- Permission issue
- UNC/network share temporarily unavailable
- Misconfigured path

What to do

- Verify `blkDirectory` exists and is readable.
- Confirm service account permissions.
- If UNC, confirm connectivity and credentials.
- Check config path escaping.

SEVERE: Unhandled error in watcher loop

What it means

An unexpected exception escaped normal per-file handling in the main polling loop.

What to do

- Inspect the stack trace immediately.
- Identify whether the error is:
 - Directory scanning
 - Control file parsing
 - mkvdk execution
 - File move / cleanup
- If repeated, Indexer may be effectively stalled until corrected or restarted.

Control file claiming / coordination

WARNING: Failed to claim control file: <controlFile>

What it means

Indexer attempted to atomically “claim” a `.index` control file and failed.

Likely causes

- Another Indexer instance already claimed it (changing it to `.processing`)
- File locked
- Permissions
- Filesystem does not support atomic move semantics as expected

What to do

- Check for multiple Indexer instances running.
- Inspect directory for `.processing` files.
- Verify filesystem supports atomic rename (especially on network shares).

WARNING: Failed to unclaim control file: <claimedPath>

What it means

Indexer attempted to release a claimed control file (e.g. after failure) and could not. (removing `.processing` from file name)

What to do

- Check permissions and locks.
- If necessary, manually move the file back or into quarantine.
- Persistent failures may require directory ownership correction.

WARNING: Failed to age-check claimed file: <path>

What it means

Indexer could not determine the age of a claimed file during stale-claim scanning.

Likely causes

- File disappeared mid-scan
- Permission issues
- Filesystem error

What to do

- Usually safe to ignore if infrequent.
- If repeated, inspect claimed directory integrity and permissions.

WARNING: Failed stale-claim age-out scan

What it means

Indexer could not complete a scan intended to clean up stale claimed files.

Operational impact

- Old claimed files may remain and block processing.

What to do

- Inspect claimed directory manually.
- Remove genuinely stale `.processing` files if safe.
- Check permissions on claimed/quarantine directories.

Control file parsing and validation

SEVERE: Problem parsing index control file; moving to quarantine and continuing: <file>

What it means

Indexer could not parse the `.index` control file into a valid internal representation.

Likely causes

- Malformed `.index` file
- Partial write by Builder
- Encoding issues
- Manual tampering

What to do

- Inspect the quarantined `.index` file.
- Check Builder logs for corresponding errors.

- Confirm Builder writes index files atomically.

WARNING: Failed to fully process Control file <file>. Control file and all related files moved to quarantine directory <dir>

What it means

Indexer started processing a control file but could not complete all steps successfully.

What to do

- Inspect quarantine contents as a set.
- Look for earlier warnings (mkvdk failures, missing files, permission issues).
- Fix root cause and re-run Builder if needed.

WARNING: Skipping editset '<editsetID>': cannot build mkvdk command (missing data in .index entry).

What it means

The `.index` entry is missing required fields to construct the mkvdk command line.

What to do

- Inspect the `.index` file content.
- Check Builder logs for warnings during index file creation.
- This usually indicates a Builder-side failure or malformed CSV input.

mkvdk execution lifecycle

WARNING: Indexing editset '<editsetID>' failed. Check mkvdk log / service log for details.

What it means

mkvdk completed but did not succeed.

What to do

- Check:
 - mkvdk stdout/stderr in Indexer logs
 - Verity logs
- Common causes:
 - Missing SGML referenced by BLK
 - Invalid BLK content
 - Collection path misconfiguration

WARNING: mkvdk[<editsetID>] timed out after <n>s; terminating process.

What it means

mkvdk exceeded the configured timeout.

What to do

- Check Verity performance and system load.
- Consider increasing timeout if large batches are expected.
- Investigate stuck mkvdk processes or IO bottlenecks.

WARNING: mkvdk[<editsetID>] did not terminate; killing forcibly.

What it means

Graceful termination failed; Indexer forcibly killed the mkvdk process.

Operational impact

- Index may be partially written.
- Associated files will be quarantined.

What to do

- Treat as abnormal.
- Inspect Verity index state.
- Investigate system-level resource issues.

WARNING: mkvdk[<editsetID>] exit code <code> (see log above for output) .

What it means

mkvdk exited normally but returned a non-success exit code.

What to do

- Look at the immediately preceding mkvdk output in logs.
- Common issues:
 - Bad BLK references
 - Missing SGML
 - Invalid collection path

WARNING: mkvdk[<editsetID>] failed to start/run.

What it means

Indexer could not launch mkvdk at all.

Likely causes

- mkvdk binary missing or not executable
- Incorrect path in config
- Permissions
- Environment/path issues

What to do

- Verify mkvdk path and execute permissions.
- Run mkvdk manually as the service account to confirm.

WARNING: mkvdk[<editsetID>] interrupted.

What it means

Indexer thread running mkvdk was interrupted (shutdown, redeploy, service stop).

What to do

- Usually benign during shutdown.
- If frequent during normal operation, investigate service restarts or thread interruptions.

File movement and cleanup

WARNING: Failed to list files in directory: <dir>

What it means

Indexer could not list files during cleanup or scanning.

Likely causes

- Directory missing
- Permission issue

Network share glitch

What to do

- Verify directory existence and permissions.
- Check for transient network issues.

WARNING: Failed to create destination directory <dir>

What it means

Indexer attempted to create a directory (processed/quarantine/zip) and failed.

What to do

- Manually create directory.
- Fix ACLs.
- Check path formatting in config.

WARNING: Error while trying to move control file: <file>

WARNING: Error while trying to move file: <file>

What it means

Indexer failed to move a control or related file (usually to processed or quarantine).

Likely causes

- File locked
- Permissions
- Destination directory missing

What to do

- Inspect filesystem state.
- Confirm no external process is touching these files.
- Verify directory structure.

WARNING: File to move is not present: <fileName>

What it means

Indexer expected a file to exist but it was already gone.

Likely causes

- Race with another process
- Manual cleanup
- Previous partial run

What to do

- Usually safe to ignore if rare.
- If frequent, ensure only one Indexer instance is active.

WARNING: trying to zip file <filePath> but couldn't find it

What it means

Indexer attempted to include a file in the zip archive but it no longer exists.

What to do

- Check that all related files were successfully produced before zipping.
- This is often secondary fallout from earlier failures.

WARNING: Failed to zip processed files to <zipFilename> (continuing).

What it means

Zippping of processed files failed, but indexing itself may still have succeeded.

Operational impact

- Archive is missing, but live index may be correct.

What to do

- Inspect zip directory permissions and disk space.
- If zips are required for audit/rollback, investigate promptly.

WARNING: Failed to remove <filePath>

WARNING: Failed to remove <filePath> file not found

WARNING: Failed to remove file <filePath>. <message>

What it means

Indexer failed to delete a file during cleanup.

Likely causes

- File locked
- Permissions
- File already removed

What to do

- Usually non-fatal.
- Periodically audit processed directories.
- If buildup occurs, investigate locking or permission problems.

Escalation and Support

When escalating issues, provide:

- Builder and/or Indexer log files
- Relevant mkvdk log files
- Affected filenames and timestamps
- Description of recent configuration changes